ICU Autonomous Triage Agent

A 3-Agent A2A System for Autonomous ICU Patient Monitoring

Samanvi Rajput

B.Tech CSE, VIT Vellore | SWE Intern, PayGlocal

2025

Abstract

ICU deterioration is time-critical: the median time from first physiological sign of deterioration to clinical intervention is over two hours in manually monitored units. Rule-based alert systems address continuous monitoring but generate high false-positive rates, leading to alarm fatigue and alert suppression. This paper presents ICU Triage Agent, a 3-agent autonomous system that monitors patient vitals streams, retrieves relevant EMR context, validates medication compliance, and produces actionable Clinical Briefs -- all without hardcoded threshold rules. The central design principle is that the LLM, not fixed rules, decides clinical significance: a Sentry agent detects deterioration trends in context, a Historian agent surfaces relevant lab results and clinical notes, and a Pharmacist agent validates whether indicated treatments were administered before broadcasting a brief. Agents communicate via an in-process A2A async message bus. Inference runs on Groq (Llama 3.3 70B, ~480 tok/s) with a full triage pipeline completing in approximately 5.7 seconds on the free tier. The system is demonstrated on a realistic CHF deterioration scenario: Patient 4B, SpO2 97 to 91, furosemide ordered but not given.

1. Problem Statement

ICU deterioration is time-critical. Manual monitoring is discontinuous -- nurses check vitals periodically, not continuously -- and the cognitive load of monitoring multiple patients simultaneously makes early detection unreliable. Rule-based alert systems (SpO2 below 92%, HR above 120) generate high false-positive rates: a COPD patient with chronic hypoxemia will trigger SpO2 alerts continuously, training staff to suppress them. The core problem is that clinical deterioration is contextual. A drop in SpO2 means something different depending on the patient's diagnosis, current medications, and lab trends. Rule-based systems cannot represent that context. LLM-based reasoning can.

This system addresses the problem with a 3-agent architecture: each agent has a focused clinical domain, a dedicated tool set, and communicates findings via a structured A2A message protocol. The output is an actionable Clinical Brief that surfaces what is wrong, why it is concerning given this patient's specific context, and what was or was not done about it.

2. System Overview

Patient 4B: 67-year-old male, congestive heart failure. SpO2 drops from 97 to 91 over 20 minutes. The pipeline proceeds as follows:

```
Patient Vitals Stream
        |
        v
[ Sentry Agent ]             <- detects deterioration trends
  MCP: get_live_vitals          SpO2, HR, MAP, RR over time
        |
        | A2A: "SpO2 trend concern"
        v
[ Historian Agent ]          <- retrieves EMR context
  MCP: query_lab_results,       BNP elevated, furosemide ordered
       fetch_clinical_notes
        |
        | A2A: "BNP elevated, furosemide ordered -- verify given"
        v
[ Pharmacist Agent ]         <- validates medication compliance
  MCP: get_active_infusions     furosemide NOT found in active orders
        |
        | A2A broadcast
        v
[ Clinical Brief ]
  "SpO2 declining in CHF patient. BNP elevated.
   Furosemide ordered but NOT administered. Recommend
   immediate review and diuretic administration."
```

3. A2A Message Protocol

Agents communicate via a structured A2AMessage dataclass carried over an in-process asyncio message bus. Each message carries: sender (agent name), recipient (agent name or 'broadcast'), message_type (query/response/clinical_brief), payload (free-form dict), and timestamp. The bus supports both directed messages and broadcast delivery.

An external message broker (Redis pub/sub, RabbitMQ) was considered and rejected for this prototype. The operational overhead -- an additional service to deploy, monitor, and secure -- is not justified for 3 agents and 4 messages per triage run. The bus interface is abstracted behind a send/subscribe API so an external broker can be substituted without changing agent code.

Patient 4B Message Flow

```
Sentry    -> Historian   [query: SpO2 trend concern]
Historian -> Sentry      [response: BNP elevated, furosemide ordered]
Historian -> Pharmacist  [query: was diuretic administered?]
Pharmacist -> broadcast  [clinical_brief: furosemide NOT given -> act now]
```

4. Agent Roles

Agent	Role	MCP Tools	LLM Task
Sentry	Vitals trend detection	get_live_vitals	Is this trend clinically concerning given the patient co
Historian	EMR context retrieval	query_lab_results, fetch_clinical_notes	What is the relevant clinical history and what was or
Pharmacist	Medication validation + brief generation	get_active_infusions	Was the indicated treatment given? Generate Clinica

5. MCP Tool Layer

Each agent has access to a narrow set of MCP tools that return structured data from the patient record. Tools are implemented as Python functions with typed signatures; in this prototype they read from the mock_data directory.

Tool	Signature	Returns
get_live_vitals	get_live_vitals(patient_id)	SpO2, HR, MAP, RR time series (last 30 min)
query_lab_results	query_lab_results(patient_id)	BNP, CBC, metabolic panel with reference ranges
fetch_clinical_notes	fetch_clinical_notes(patient_id)	Structured clinical history, diagnoses, prior orders
get_active_infusions	get_active_infusions(patient_id)	Current medication orders and administration records

The mock data layer uses a realistic Patient 4B scenario: CHF diagnosis, SpO2 declining from 97 to 91, BNP elevated at 840 pg/mL, furosemide 40mg ordered but not appearing in the active infusions record. This scenario exercises all three agent capabilities and produces an unambiguous Clinical Brief. Real EMR integration (Epic FHIR R4, Cerner) requires hospital IT approval and HIPAA compliance infrastructure not in scope for a research prototype; the tool interface is designed for FHIR swap-in.

6. LLM Design

Each agent is instantiated with its own system prompt that defines its clinical domain, tool access, and output format. The LLM -- Llama 3.3 70B served via Groq's LPU inference hardware -- is called once per agent per triage run. The OpenAI SDK is used with the base URL pointed at the Groq endpoint, making the model substitutable without changing agent code.

Groq's LPU hardware delivers approximately 480 tokens/second on Llama 3.3 70B, with a mean time-to-first-token of ~210ms. This is roughly 10x faster than equivalent GPU inference and is the primary reason Groq was chosen over a GPU-hosted open model. The Groq free tier is rate-limited to 30 requests/minute. The orchestrator inserts a 2-second delay between agent LLM calls to stay within this limit, adding ~4 seconds to each triage run. A production deployment would use the paid tier or implement exponential backoff with retry.

7. Design Decisions

Decision	Chosen	Alternative	Reason
Agent communication	A2A async bus	Single monolithic prompt	Domain separation, focused context per agent
Clinical reasoning	LLM judgment	Hardcoded thresholds	Patient-specific context, handles edge cases
Inference	Groq Llama 3.3 70B	GPT-4 / Claude	~480 tok/s speed, open model, no vendor lock-in
Message broker	In-process asyncio	Redis / RabbitMQ	No extra service, sufficient for prototype scale
UI	Streamlit	React + WebSocket	Rapid prototyping, 3-column dashboard in minim
Patient data	Mock scenario	Live EMR (FHIR)	HIPAA compliance not in scope for research pro

8. Performance Benchmarks

MacBook M2 8GB, Groq free tier, Llama 3.3 70B.

Per-Agent Latency

Agent	LLM Calls	Mean LLM Latency	Total (incl. 2s delay)
Sentry	1	~680ms	~680ms
Historian	1	~920ms	~2,920ms
Pharmacist	1	~1,100ms	~4,100ms
Full triage run	3	--	~5.7s

The 2-second inter-call delay accounts for approximately 4 seconds of the total 5.7-second runtime. Without rate limiting, the full pipeline completes in approximately 2.7 seconds. Message bus overhead is under 1ms per message -- less than 0.1% of total runtime.

LLM Performance (Groq)

Metric	Value
Model	Llama 3.3 70B
Tokens/second (Groq LPU)	~480 tok/s
Mean tokens per agent call	~340 tok
Mean time-to-first-token	~210ms

9. Engineering Challenges

Groq rate limiting with 3 sequential LLM calls

The Groq free tier allows 30 requests/minute. A single triage run makes 3 LLM calls minimum -- at continuous monitoring frequency this hits the limit within minutes. The orchestrator inserts a 2-second delay between calls, preventing 429 errors that would leave a Clinical Brief incomplete. A production deployment requires the paid tier or exponential backoff with retry logic.

LLM non-determinism in clinical context

The same vitals input produces slightly different Clinical Brief wording across runs. For a research prototype this is acceptable. For clinical deployment, the Pharmacist agent's system prompt includes explicit output format constraints and a mandatory medication verification step before any broadcast -- ensuring the actionable conclusion is consistent even if phrasing varies.

A2A message ordering under async execution

With asyncio, concurrent agent execution does not guarantee message delivery order. Early versions had the Pharmacist generating a Clinical Brief before the Historian completed EMR retrieval, producing briefs with incomplete context. The orchestrator now enforces a sequential pipeline (Sentry, then Historian, then Pharmacist), trading parallel throughput for correctness.

Context accumulation across agent turns

Each agent maintains its own conversation context across triage runs. Without pruning, this context grows unbounded and eventually exceeds the model's context window. Context is pruned to the last N turns per agent after each triage run, keeping working memory bounded while preserving recent history.

10. Future Work

Near-term

- Streaming Clinical Brief -- SSE token streaming to Streamlit for real-time output
- Multi-patient support -- orchestrator handles multiple patient IDs concurrently
- Configurable agent roster -- add/remove agents without modifying orchestrator code

Medium-term

- FHIR integration -- replace mock_data with real HL7 FHIR R4 endpoints
- Escalation routing -- webhook paging of on-call physician at severity threshold
- Audit log -- immutable record of every A2A message and Clinical Brief

Long-term

- Local inference -- replace Groq with Ollama for air-gapped hospital deployments
- SOFA/NEWS2 scoring layer -- dedicated agent for standardized severity scores
- Multi-hospital federation -- shared anonymized deterioration patterns

---

11. Closing

The core insight of this system is that ICU alarm fatigue is not a sensitivity problem -- it is a context problem. Threshold-based rules fire on individual values without knowing who the patient is. An LLM that reasons over combined vitals, lab trends, and medication history does what a clinician does: it asks whether this finding is concerning for this patient, right now, given what we know. The 3-agent A2A architecture distributes that reasoning across focused domains -- the same way an ICU team does -- and produces a brief that tells the clinician not just what happened, but what to do.